Assignment 3 - Handwritten Digit Classification using SVM

Classify handwritten digits (0-9) using linear and RBF-kernel SVMs.

Dataset: scikit-learn `load_digits` *(8 x 8 images, 64 features).*

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
)

RANDOM_STATE = 42
```

1. Data understanding and cleaning

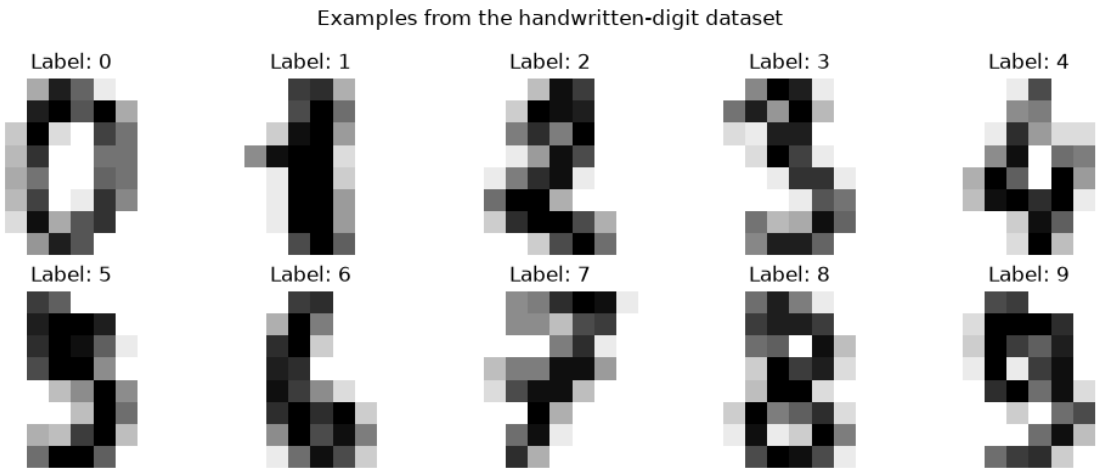
In [2]:
```
digits = load_digits()
X = digits.data
y = digits.target

print("Feature matrix shape:", X.shape)
print("Image shape:", digits.images.shape[1:])
print("Classes:", np.unique(y))
print("Pixel-value range:", X.min(), "to", X.max())
print("Missing values:", np.isnan(X).sum())
```

```
Feature matrix shape: (1797, 64)
Image shape: (8, 8)
Classes: [0 1 2 3 4 5 6 7 8 9]
Pixel-value range: 0.0 to 16.0
Missing values: 0
```

In [3]:
```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for ax, image, label in zip(axes.ravel(), digits.images[:10], y[:10]):
    ax.imshow(image, cmap="gray_r")
    ax.set_title(f"Label: {label}")
    ax.axis("off")

plt.suptitle("Examples from the handwritten-digit dataset")
plt.tight_layout()
plt.show()
```



2. Data preparation

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=RANDOM_STATE,
    stratify=y,
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
print("Scaled training mean (first feature):", round(X_train_scaled[:,
0].mean(), 6))
print(
    "Scaled training standard deviation (first feature):",
    round(X_train_scaled[:, 0].std(), 6),
)
```

```
Training set: (1257, 64)
Test set: (540, 64)
Scaled training mean (first feature): 0.0
Scaled training standard deviation (first feature): 0.0
```

3. Linear SVM model

In [5]:
```
linear_model = SVC(kernel="linear", C=1.0)
linear_model.fit(X_train_scaled, y_train)
y_linear_pred = linear_model.predict(X_test_scaled)

linear_accuracy = accuracy_score(y_test, y_linear_pred)
print(f"Linear SVM accuracy: {linear_accuracy * 100:.2f}%")
```

```
Linear SVM accuracy: 97.96%
```

4. Non-linear RBF SVM model

In [6]:
```
rbf_model = SVC(kernel="rbf", C=10.0, gamma="scale")
rbf_model.fit(X_train_scaled, y_train)
y_rbf_pred = rbf_model.predict(X_test_scaled)

rbf_accuracy = accuracy_score(y_test, y_rbf_pred)
print(f"RBF SVM accuracy: {rbf_accuracy * 100:.2f}%")
```

```
RBF SVM accuracy: 98.15%
```

5. Hyperparameter tuning

In [7]:
```
tuning_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf")),
])

param_grid = {
    "svm__C": [1, 10, 30],
    "svm__gamma": ["scale", 0.001, 0.01],
}

grid_search = GridSearchCV(
    tuning_pipeline,
    param_grid,
    cv=3,
    scoring="accuracy",
    n_jobs=-1,
)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
y_tuned_pred = best_model.predict(X_test)
tuned_accuracy = accuracy_score(y_test, y_tuned_pred)

print("Best parameters:", grid_search.best_params_)
print(f"Best cross-validation accuracy: {grid_search.best_score_ *
100:.2f}%")
print(f"Tuned RBF SVM test accuracy: {tuned_accuracy * 100:.2f}%")
```

```
Best parameters: {'svm__C': 10, 'svm__gamma': 'scale'}
Best cross-validation accuracy: 98.25%
Tuned RBF SVM test accuracy: 98.15%
```

6. Model evaluation

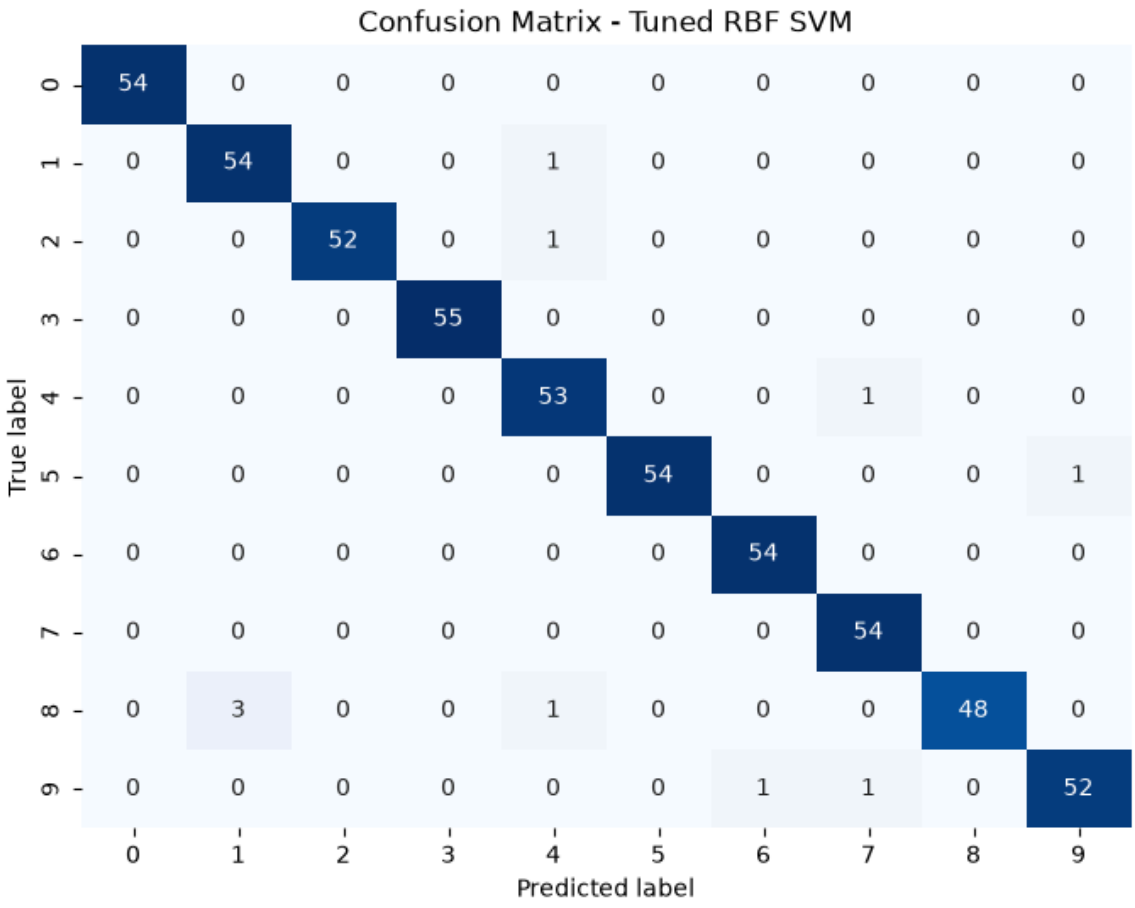
In [8]:
```
print("Classification report for the tuned RBF SVM:\n")
print(classification_report(y_test, y_tuned_pred))

confusion = confusion_matrix(y_test, y_tuned_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(confusion, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("Confusion Matrix - Tuned RBF SVM")
```

```
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.show()
```

```
Classification report for the tuned RBF SVM:
```

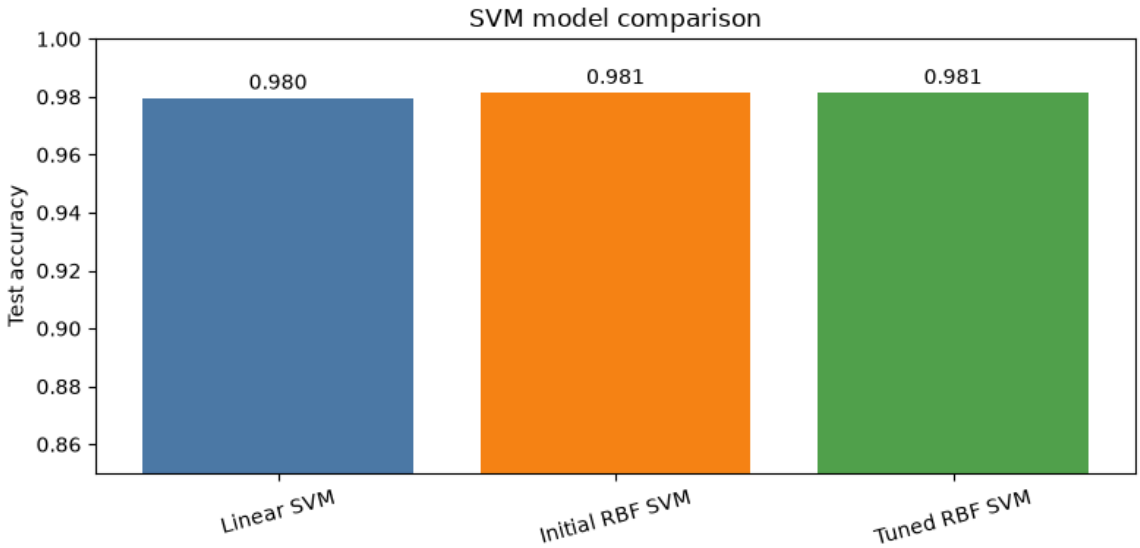
	precision	recall	f1-score	support
0	1.00	1.00	1.00	54
1	0.95	0.98	0.96	55
2	1.00	0.98	0.99	53
3	1.00	1.00	1.00	55
4	0.95	0.98	0.96	54
5	1.00	0.98	0.99	55
6	0.98	1.00	0.99	54
7	0.96	1.00	0.98	54
8	1.00	0.92	0.96	52
9	0.98	0.96	0.97	54
accuracy			0.98	540
macro avg	0.98	0.98	0.98	540
weighted avg	0.98	0.98	0.98	540



In [9]:
```
comparison = {
    "Linear SVM": linear_accuracy,
    "Initial RBF SVM": rbf_accuracy,
    "Tuned RBF SVM": tuned_accuracy,
}

plt.figure(figsize=(8, 4))
```

```
bars = plt.bar(
    comparison.keys(),
    comparison.values(),
    color=["#4c78a8", "#f58518", "#54a24b"],
)
plt.ylim(0.85, 1.0)
plt.ylabel("Test accuracy")
plt.title("SVM model comparison")
plt.xticks(rotation=15)
for bar, score in zip(bars, comparison.values()):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        score + 0.003,
        f"{score:.3f}",
        ha="center",
    )
plt.tight_layout()
plt.show()
```



7. Visualize predictions

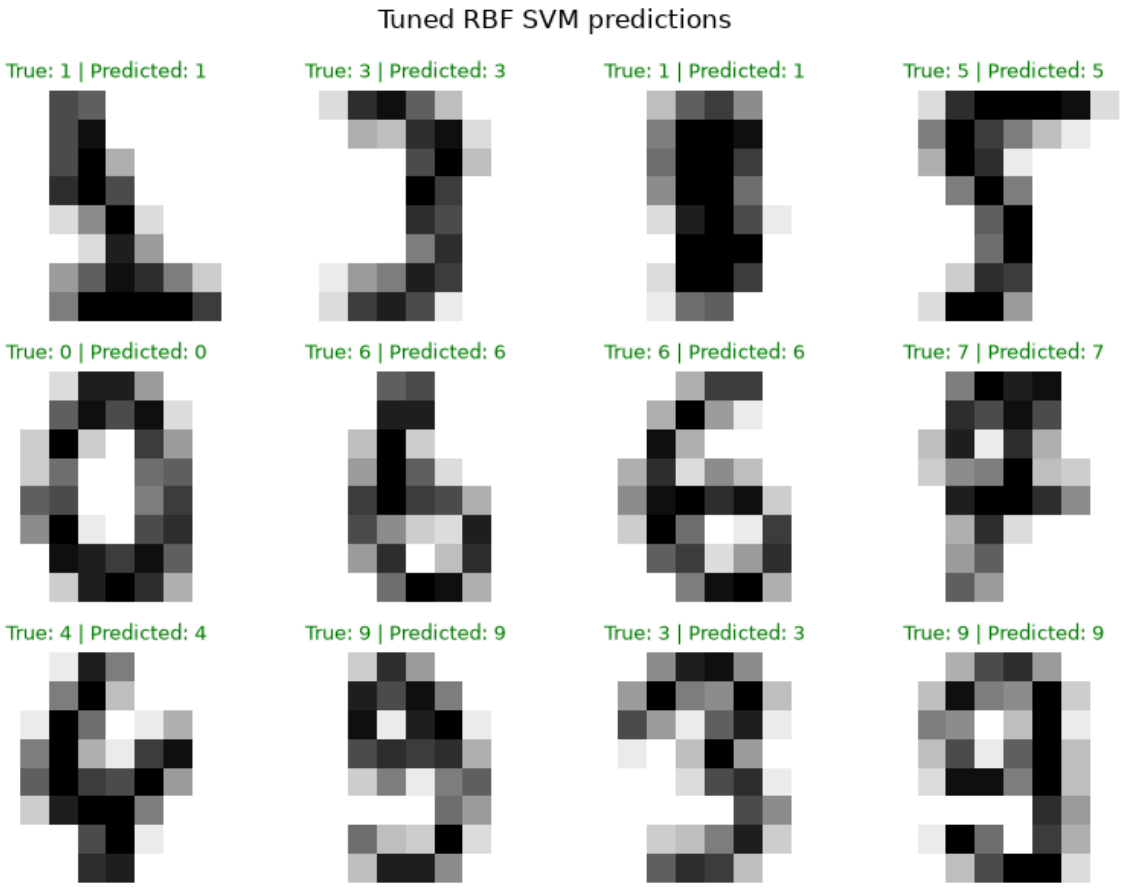
In [10]:

```
sample_count = 12
sample_indices = np.arange(sample_count)

fig, axes = plt.subplots(3, 4, figsize=(8, 6))
for ax, index in zip(axes.ravel(), sample_indices):
    ax.imshow(X_test[index].reshape(8, 8), cmap="gray_r")
    predicted = y_tuned_pred[index]
    actual = y_test[index]
    colour = "green" if predicted == actual else "red"
    ax.set_title(
        f"True: {actual} | Predicted: {predicted}",
        color=colour,
        fontsize=9,
    )
    ax.axis("off")

plt.suptitle("Tuned RBF SVM predictions")
```

```
plt.tight_layout()
plt.show()
```



Assignment 4 - K-Means Clustering on Iris Dataset

Perform clustering analysis using K-Means and determine the number of clusters with the elbow method.

Dataset: `prac1/Iris.csv` .

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
```

1. Load and prepare the data

In [2]:
```
df = pd.read_csv("./Iris.csv")
df = df.drop(columns=["Id"], errors="ignore")

X = df.drop(columns=["Species"])
y = df["Species"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Dataset shape:", X.shape)
print("Missing values:", X.isna().sum().sum())
print("Classes:", y.unique())
```

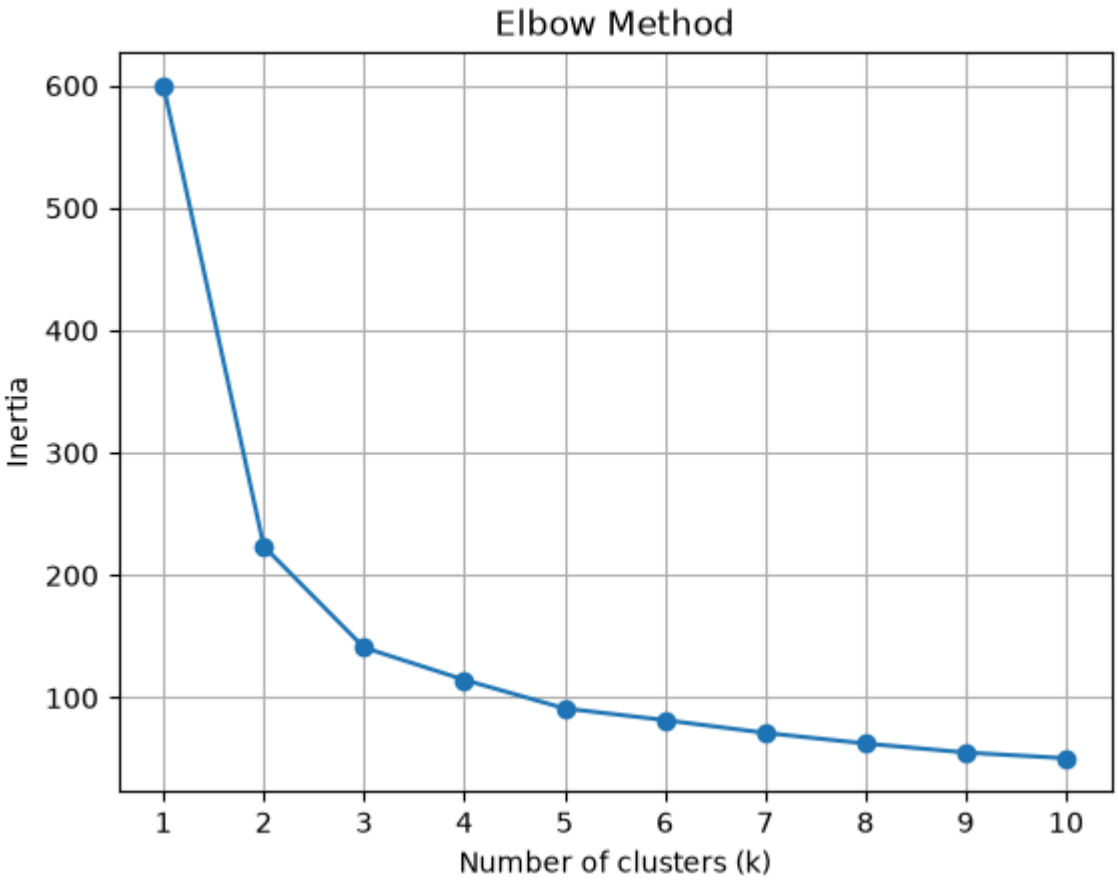
```
Dataset shape: (150, 4)
Missing values: 0
Classes: <StringArray>
['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
Length: 3, dtype: str
```

2. Elbow method

In [3]:
```
inertia = []
k_values = range(1, 11)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    model.fit(X_scaled)
    inertia.append(model.inertia_)
```

```
plt.plot(k_values, inertia, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```



3. Build the K-Means model

In [4]:
```
kmeans = KMeans(n_clusters=3, random_state=RANDOM_STATE, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

print("Cluster sizes:", np.bincount(clusters))
print("Silhouette score:", round(silhouette_score(X_scaled, clusters), 4))
print("Adjusted Rand index:", round(adjusted_rand_score(y, clusters), 4))
```

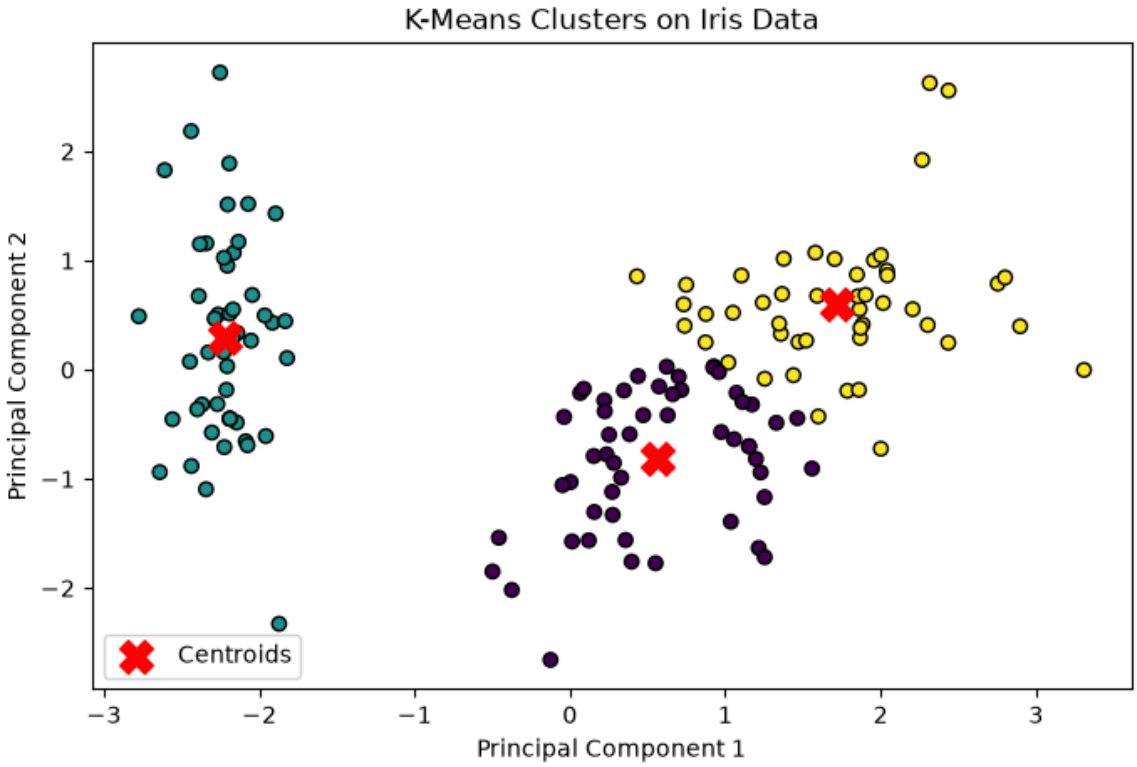
```
Cluster sizes: [53 50 47]
Silhouette score: 0.459
Adjusted Rand index: 0.6201
```

4. Visualize the clusters

In [5]:
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis",
```

```
edgecolor="k")
plt.scatter(
    pca.transform(kmeans.cluster_centers_)[:, 0],
    pca.transform(kmeans.cluster_centers_)[:, 1],
    marker="X",
    s=200,
    c="red",
    label="Centroids",
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clusters on Iris Data")
plt.legend()
plt.show()
```



Conclusion

The elbow in the inertia curve occurs around k = 3, so three clusters are selected. K-Means groups the Iris samples into three clusters that correspond reasonably well to the three species.

Assignment 5 - Random Forest Classifier

Predict car safety using a Random Forest classifier.

Dataset: Car Evaluation dataset.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

RANDOM_STATE = 42
```

1. Load and prepare the data

In [2]:
```
columns = ["buying", "maint", "doors", "persons", "lug_boot", "safety",
"class"]
df = pd.read_csv("car.data", names=columns)

X = df.drop(columns=["class"])
y = df["class"]

print("Dataset shape:", df.shape)
print("Missing values:", df.isna().sum().sum())
print("Classes:", y.value_counts().to_dict())
```

```
Dataset shape: (1728, 7)
Missing values: 0
Classes: {'unacc': 1210, 'acc': 384, 'good': 69, 'vgood': 65}
```

2. Split the data

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y
)

categorical_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    [("onehot", OneHotEncoder(handle_unknown="ignore"),
categorical_features)]
)

model = Pipeline([
    ("preprocessor", preprocessor),
```

```
    ("classifier", RandomForestClassifier(
        n_estimators=200, random_state=RANDOM_STATE, class_weight="balanced"
    )),
])
```

3. Train the Random Forest model

In [4]:
```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 96.92%

Classification Report:

              precision    recall  f1-score   support

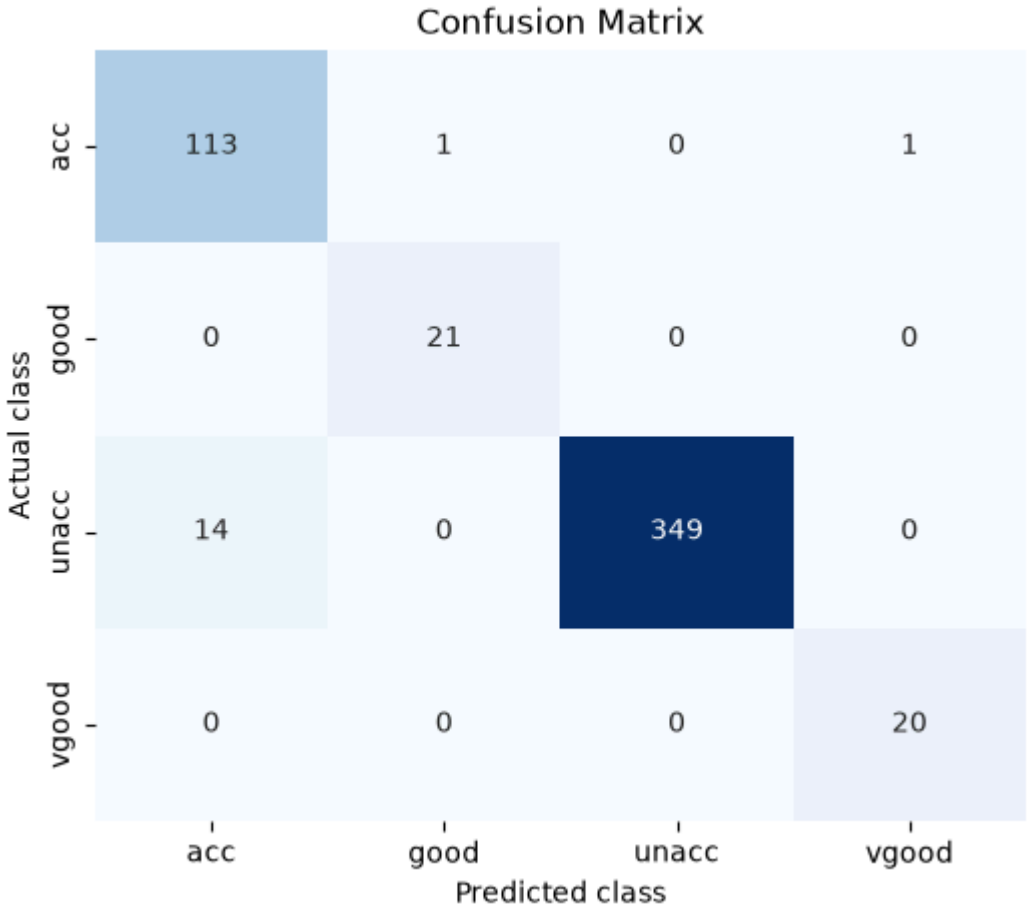
         acc       0.89      0.98      0.93       115
        good       0.95      1.00      0.98        21
       unacc       1.00      0.96      0.98       363
       vgood       0.95      1.00      0.98        20

    accuracy                           0.97       519
   macro avg       0.95      0.99      0.97       519
weighted avg       0.97      0.97      0.97       519
```

4. Evaluate the model

In [5]:
```
labels = model.named_steps["classifier"].classes_
confusion = confusion_matrix(y_test, y_pred, labels=labels)

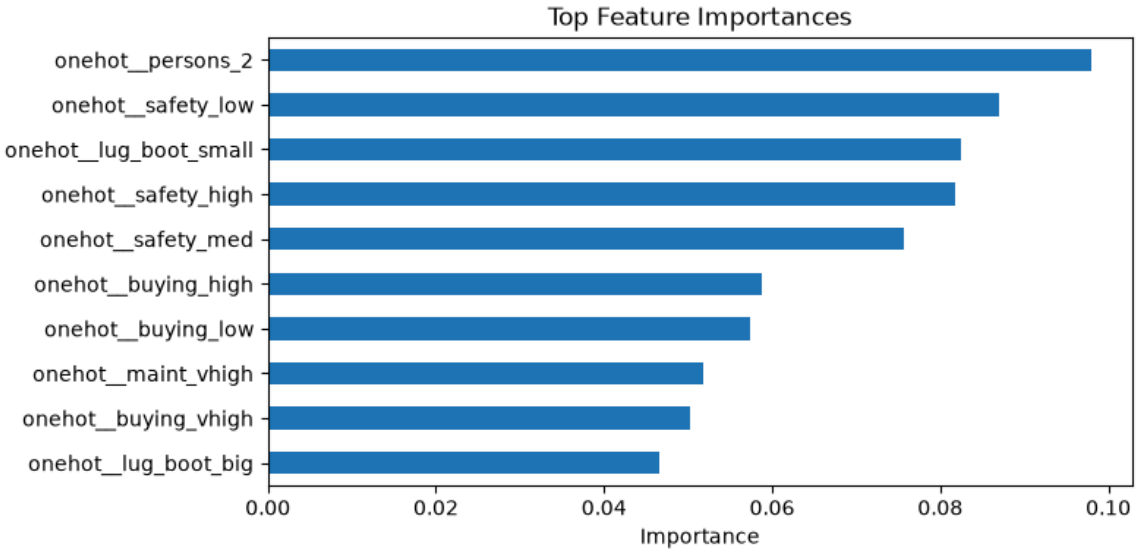
plt.figure(figsize=(6, 5))
sns.heatmap(confusion, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted class")
plt.ylabel("Actual class")
plt.title("Confusion Matrix")
plt.show()
```



In [6]:

```
feature_names = model.named_steps["preprocessor"].get_feature_names_out()
importance = model.named_steps["classifier"].feature_importances_
top_features = (
    pd.Series(importance, index=feature_names)
    .sort_values(ascending=False)
    .head(10)
)

top_features.sort_values().plot(kind="barh", figsize=(8, 4))
plt.xlabel("Importance")
plt.title("Top Feature Importances")
plt.tight_layout()
plt.show()
```



Conclusion

The Random Forest model classifies car-evaluation categories from the six categorical features. One-hot encoding converts the categories into numeric features before training.

Assignment 6 - Tic-Tac-Toe using Reinforcement Learning

Build and train a Q-learning agent to play Tic-Tac-Toe.

1. Setting up the environment

In [1]:
```
import random
import numpy as np
import matplotlib.pyplot as plt

RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)
```

2. Defining the Tic-Tac-Toe game

In [2]:
```
class TicTacToe:
    def __init__(self):
        self.winning_lines = (
            (0, 1, 2), (3, 4, 5), (6, 7, 8),
            (0, 3, 6), (1, 4, 7), (2, 5, 8),
            (0, 4, 8), (2, 4, 6),
        )
        self.reset()

    def reset(self):
        self.board = [0] * 9
        return tuple(self.board)

    def available_actions(self):
        return [i for i, value in enumerate(self.board) if value == 0]

    def winner(self):
        for a, b, c in self.winning_lines:
            if (
                self.board[a] != 0
                and self.board[a] == self.board[b] == self.board[c]
            ):
                return self.board[a]
        return 0

    def step(self, action, player):
        if action not in self.available_actions():
            raise ValueError("Action must be an available board position")
        self.board[action] = player
        winner = self.winner()
        done = winner != 0 or not self.available_actions()
        return tuple(self.board), winner, done
```

```
    def display(self):
        symbols = {0: " ", 1: "X", -1: "O"}
        for row in range(0, 9, 3):
            print(" | ".join(symbols[x] for x in self.board[row:row + 3]))
            if row < 6:
                print("---------")
```

3. Building the reinforcement learning model

In [3]:
```
class QLearningAgent:
    def __init__(self, learning_rate=0.2, discount=0.95, epsilon=1.0):
        self.q_table = {}
        self.learning_rate = learning_rate
        self.discount = discount
        self.epsilon = epsilon

    def values(self, state):
        if state not in self.q_table:
            self.q_table[state] = np.zeros(9)
        return self.q_table[state]

    def choose_action(self, state, actions, explore=True):
        if explore and random.random() < self.epsilon:
            return random.choice(actions)
        values = self.values(state).copy()
        values[[i for i in range(9) if i not in actions]] = -np.inf
        return int(np.argmax(values))

    def update(self, state, action, reward, next_state, next_actions, done):
        current = self.values(state)[action]
        future = 0 if done else np.max(self.values(next_state)[next_actions])
        target = reward + self.discount * future
        self.q_table[state][action] += self.learning_rate * (target -
current)
```

4. Training the model

In [4]:
```
def train_agent(episodes=20000):
    agent = QLearningAgent()
    results = []

    for episode in range(episodes):
        game = TicTacToe()
        state = game.reset()
        previous_state = None
        previous_action = None

        while True:
            actions = game.available_actions()
            action = agent.choose_action(state, actions)
            next_state, winner, done = game.step(action, 1)

            if done:
                reward = 1 if winner == 1 else 0
                agent.update(state, action, reward, next_state, [], True)
```

```
                results.append(winner)
                break

            opponent_action = random.choice(game.available_actions())
            next_state, winner, done = game.step(opponent_action, -1)
            reward = -1 if winner == -1 else (0 if done else 0)
            agent.update(
                state, action, reward, next_state,
                game.available_actions(), done
            )

            if done:
                results.append(winner)
                break
            state = next_state

        agent.epsilon = max(0.05, agent.epsilon * 0.9997)

    return agent, results

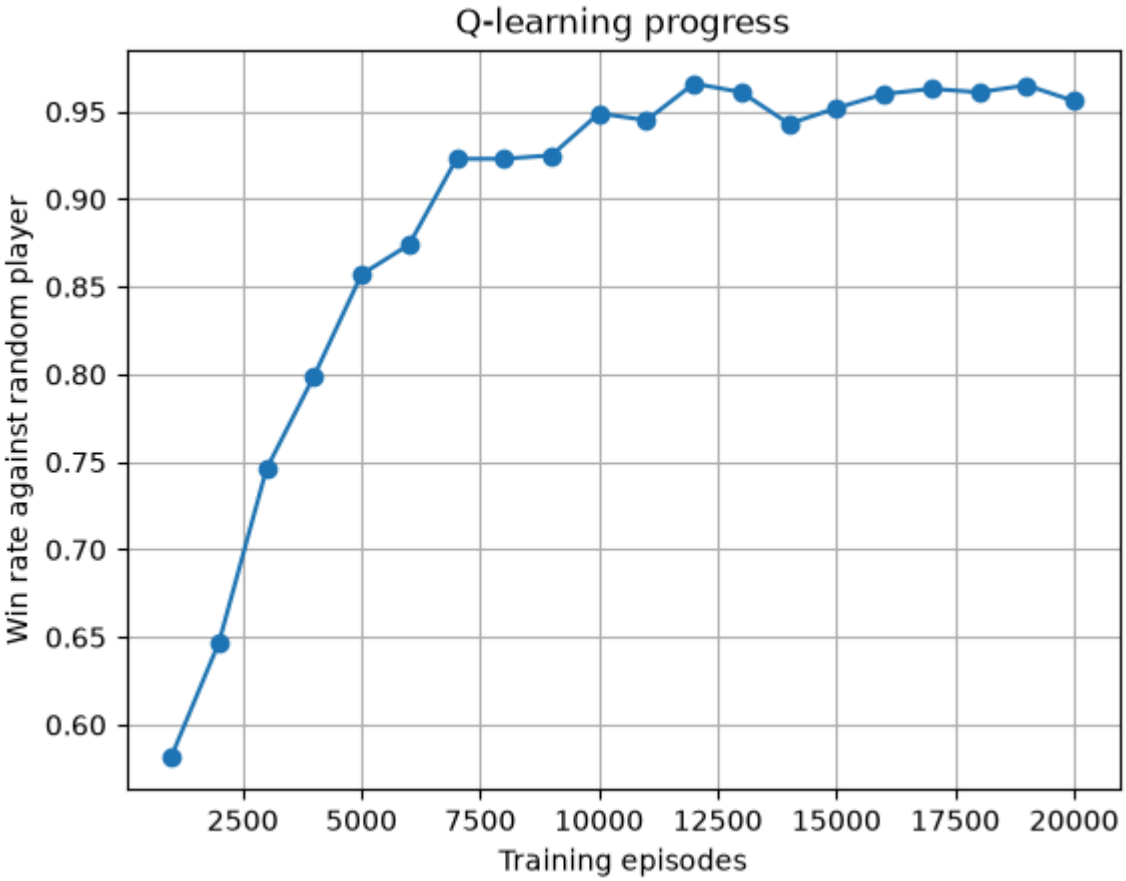
agent, training_results = train_agent()
print("Learned states:", len(agent.q_table))
print("Final exploration rate:", round(agent.epsilon, 3))
```

```
Learned states: 2161
Final exploration rate: 0.05
```

In [5]:

```
window = 1000
win_rates = [
    np.mean(np.array(training_results[i:i + window]) == 1)
    for i in range(0, len(training_results), window)
]

plt.plot(range(window, len(training_results) + 1, window), win_rates,
marker="o")
plt.xlabel("Training episodes")
plt.ylabel("Win rate against random player")
plt.title("Q-learning progress")
plt.grid(True)
plt.show()
```



5. Testing the model

In [6]:
```python
def test_agent(agent, games=1000):
    outcomes = {1: 0, -1: 0, 0: 0}
    for _ in range(games):
        game = TicTacToe()
        state = game.reset()
        while True:
            action = agent.choose_action(
                state, game.available_actions(), explore=False
            )
            state, winner, done = game.step(action, 1)
            if done:
                outcomes[winner] += 1
                break
            action = random.choice(game.available_actions())
            state, winner, done = game.step(action, -1)
            if done:
                outcomes[winner] += 1
                break
    return outcomes

test_results = test_agent(agent)
print("Test results:", test_results)
print("Test win rate:", round(test_results[1] / 1000 * 100, 2), "%")
```

```
Test results: {1: 983, -1: 0, 0: 17}
Test win rate: 98.3 %
```

In [7]:
```
game = TicTacToe()
state = game.reset()

while True:
    action = agent.choose_action(state, game.available_actions(),
explore=False)
    state, winner, done = game.step(action, 1)
    if done:
        break
    opponent_action = random.choice(game.available_actions())
    state, winner, done = game.step(opponent_action, -1)
    if done:
        break

game.display()
result = (
    "Agent wins" if winner == 1
    else "Random player wins" if winner == -1
    else "Draw"
)
print("Result:", result)
```

```
X | X | X
---------
O |   |
---------
O |   |
Result: Agent wins
```

Conclusion

The Q-learning agent learns action values from rewards and improves its Tic-Tac-Toe performance through repeated games against a random opponent.